CLASE 6 · 3 HS

Arreglos y Combinacionales

Guardar tablas en arreglos de C. Al salir, el entrenador muestra números en un display de 7 segmentos con un decodificador que es… **una sola línea de código**.

Programación en C con ESP32 · Sistemas Lógicos · Universidad Nacional de Santiago del Estero · 2026

HOY

Ocho bloques: de C puro a la tabla de verdad

1 · 10 min

Muchas variables iguales

2 · 35 min

Arreglos desde cero

3 · 15 min

Combinacional = tabla

4 · 20 min

Decodificador 2 → 4

5 · 30 min

Display de 7 segmentos

6 · 15 min

Multiplexor 4 → 1

7 · 15 min

Función de 3 variables

8 · 40 min

Ejercicios y TP

Siete días, siete variables

```
int tempLunes = 31;
int tempMartes = 34;
int tempMiercoles = 29;
//  ... y así hasta el domingo

promedio = (tempLunes + tempMartes +  ...) / 7;
```

Para pensar

¿Y si fueran los **365 días** del año?

Hace falta **una sola variable que guarde muchos valores**, y elegir cuál con un número. Eso es un **arreglo** (*array*).

Una fila de cajitas numeradas

```
int temperaturas[7] = {31, 34, 29, 38, 36, 27, 33};
```

[0]	[1]	[2]	[3]	[4]	[5]	[6]
31	34	29	38	36	27	33

`int`

El **tipo**: todos los elementos son iguales.

`temperaturas`

El **nombre** del arreglo.

`[7]`

El **tamaño**: fijo, no crece ni se achica.

`{31, 34, …}`

Los **valores iniciales**, en orden.

El índice empieza en 0

```
temperaturas[0]      // 31: el PRIMERO
temperaturas[6]      // 33: el ÚLTIMO
for (int i = 0; i < DIAS; i++) {
  Serial.printf("temperaturas[%d] = %d\n", i, temperaturas[i]);
}
```

0 … 6

Índices válidos de un arreglo de 7. El último es **tamaño − 1**.

i < DIAS

No **i <= DIAS**: así i toma justo los valores 0 a 6.

¿Por qué 0?

Es cuántos lugares correrse desde el principio. Como **L0** y el **bit 0**.

Dos recetas clásicas: suma y máximo

```
int suma = 0;
for (int i = 0; i < DIAS; i++) {
  suma = suma + temperaturas[i];
}
int promedio = suma / DIAS;
```

Para hacer

Predecir el promedio antes de correrlo. Después, agregarle el **mínimo**.

```
int maxima = temperaturas[0];
int diaMaximo = 0;
for (int i = 1; i < DIAS; i++) {
  if (temperaturas[i] > maxima) {
    maxima = temperaturas[i];
    diaMaximo = i;
  }
}
// suponer que el primero es el
// mayor y comparar con el resto
```

Salirse del arreglo: C no avisa

```
int temperaturas[7] = {31, 34, 29, 38, 36, 27, 33};
Serial.printf("%d\n", temperaturas[7]);  // ¡NO EXISTE! Imprime cualquier cosa
```

Qué puede pasar

Un número basura, otra variable que cambia "sola", o que el ESP32 se reinicie.

Regla de oro

Antes de usar **arreglo[i]**: i entre 0 y tamaño − 1. Error común: **i <= 7**.

Tablas const

Con **const int LEDS[8]**, escribir LEDS[0] = 5; no compila. Protección gratis.

”C confía en el programador. Demasiado.”



Un circuito combinacional es una tabla

TABLA DE VERDAD

entrada	salida
0	0001
1	0010
2	0100
3	1000



ARREGLO EN C

índice	contenido
0	0001
1	0010
2	0100
3	1000

Así es una ROM

Las entradas van a las líneas de **dirección**; las salidas salen por las de **datos**. Las **FPGA** están hechas de miles de **LUT**: tablas de verdad guardadas en memoria.

```
salida = TABLA[entrada];
```

La **entrada** es el índice; la **salida** es el contenido. Una tabla de búsqueda (*lookup table*, LUT).

4 · DECODIFICADOR 2 → 4

Se enciende solo la salida elegida

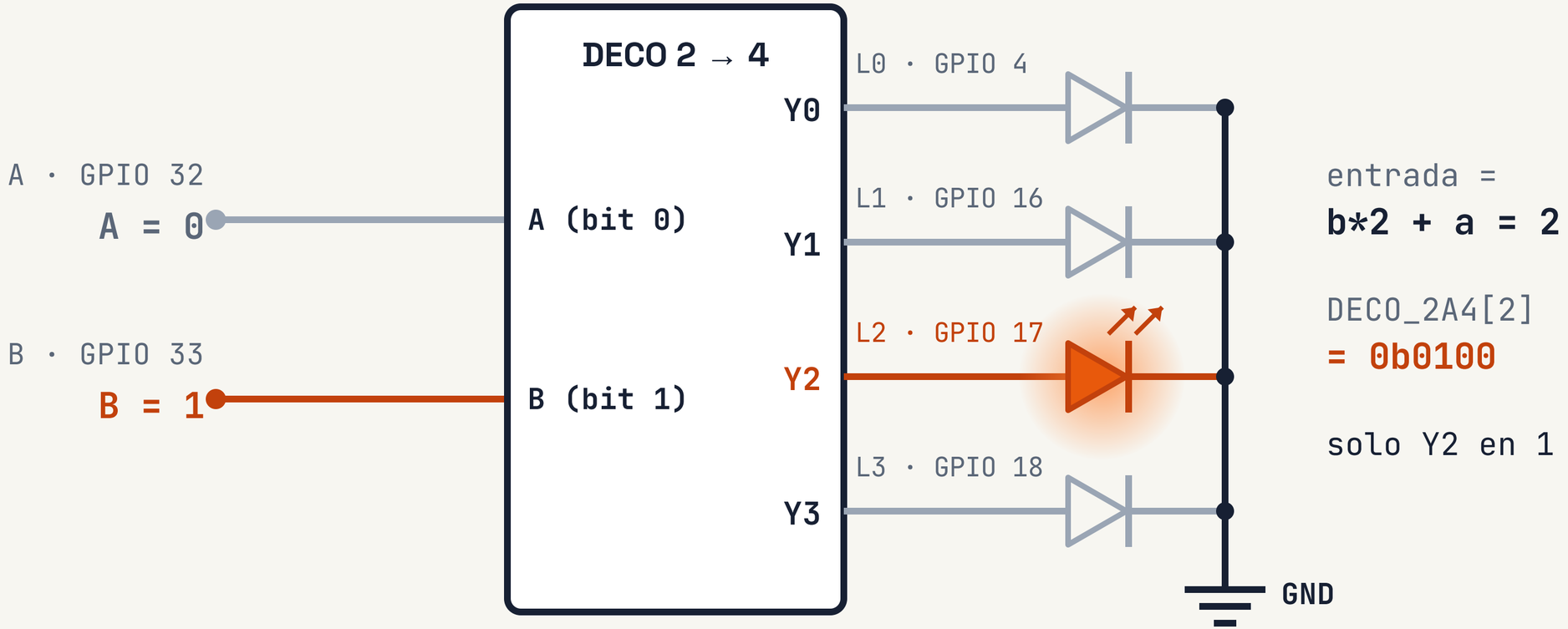
B	A	Entrada	Y3	Y2	Y1	Y0
0	0	0	0	0	0	1
0	1	1	0	0	1	0
1	0	2	0	1	0	0
1	1	3	1	0	0	0

Primero, **juntar las entradas en un número**: cada bit por su peso.

```
bool a = (digitalRead(BOTON_A) == LOW);
bool b = (digitalRead(BOTON_B) == LOW);
int entrada = b * 2 + a;
// B pesa 2, A pesa 1
```

En la placa: A = bit 0, B = bit 1, salidas en **L0..L3**. Con la Clase 5 también vale **(b << 1) | a**.

Apretado B: entrada 2, enciende L2



Con if o con tabla

FORMA 1 · COMO LAS ECUACIONES

```
uint8_t decoConIf(bool a, bool b) {
  if (!b && !a) {
    return 0b0001;
  } else if (!b && a) {
    return 0b0010;
  } else if (b && !a) {
    return 0b0100;
  } else {
    return 0b1000;
  }
}
```

FORMA 2 · COMO UNA ROM

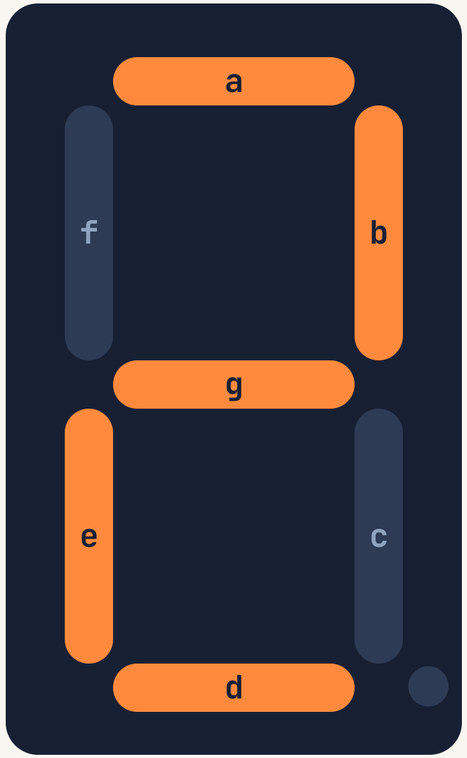
```
const uint8_t DECO_2A4[4] =
  {0b0001, 0b0010, 0b0100, 0b1000};

uint8_t decoConTabla(bool a, bool b) {
  int entrada = b * 2 + a;
  return DECO_2A4[entrada];
}
```

Cambiar la función = **cambiar números** en la tabla. ¿Y un 3 → 8? Pista: **1 << entrada**.



Un dígito es un byte, un bit por segmento



7 LEDs con forma de rayita, **a** a **g**, más el punto **dp**. Cátodo común: un **1** en el pin enciende el segmento. El **2** usa a, b, g, e, d:

bit	7	6	5	4	3	2	1	0
segmento	dp	g	f	e	d	c	b	a
valor	0	1	0	1	1	0	1	1

→ **0b01011011**

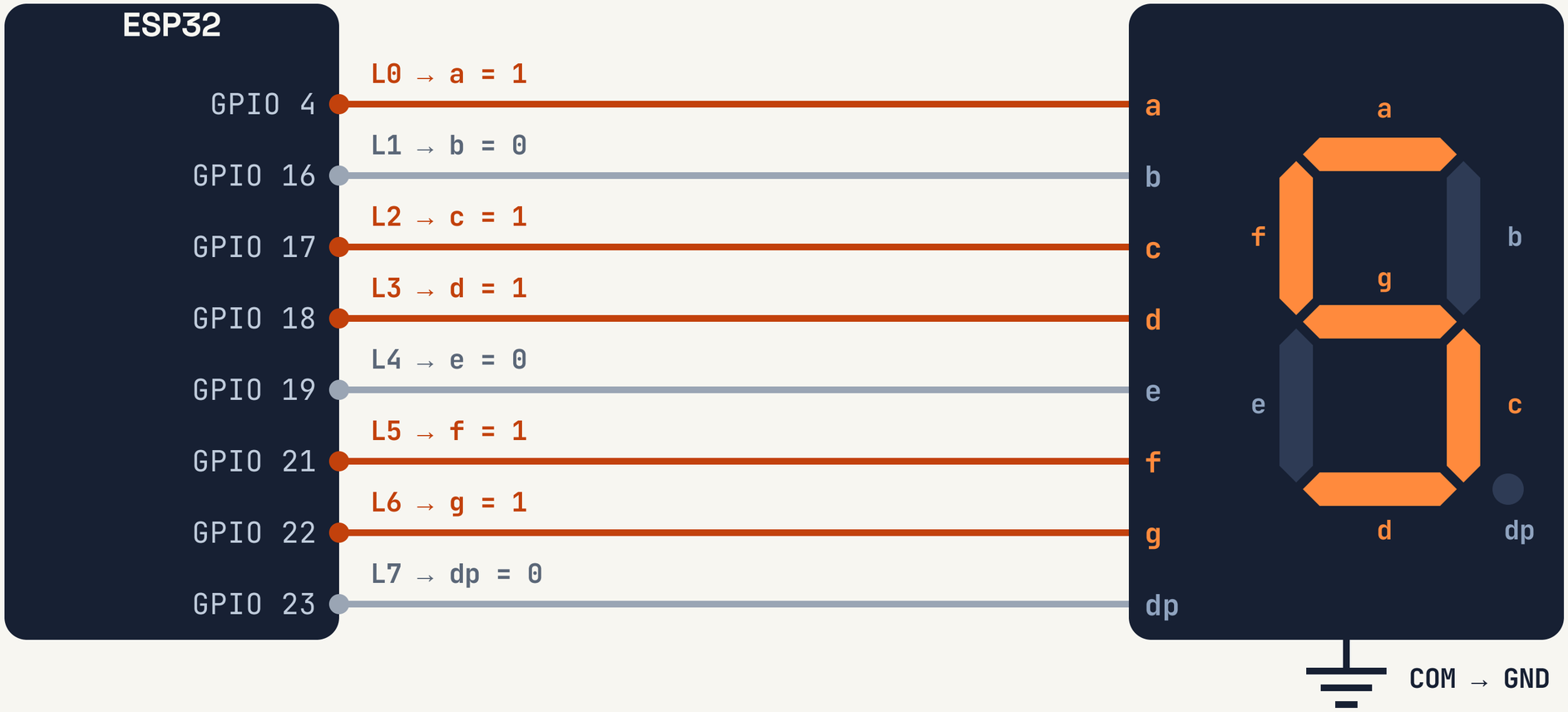
En la placa de cátedra: **a = L0, b = L1, …, g = L6, dp = L7**. Lo que manda mostrarByte se ve a la vez en los LEDs y en el display.

La tabla de los 16 dígitos hexa

```
const uint8_t SEGMENTOS[16] = {
  0b00111111,  // 0: a b c d e f
  0b00000110,  // 1: b c
  0b01011011,  // 2: a b d e g
  0b01001111,  // 3: a b c d g
  0b01100110,  // 4: b c f g
  0b01101101,  // 5: a c d f g
  0b01111101,  // 6: a c d e f g
  0b00000111,  // 7: a b c
  0b01111111,  // 8: todos
  0b01101111,  // 9: a b c d f g
  0b01110111,  // A: a b c e f g
  0b01111100,  // b: c d e f g
  0b00111001,  // C: a d e f
  0b01011110,  // d: b c d e g
  0b01111001,  // E: a d e f g
  0b01110001   // F: a e f g
};
```

`mostrarByte(SEGMENTOS[n]);` El decodificador completo es **esta línea**.

Cada segmento es un LED en un pin (n = 5)



SEGMENTOS[5]

0b01101101

dp g f e d c b a

cátodo común

Agregar el display al proyecto propio

Ya listos

Los ejemplos con display traen en **wokwi/** el diagram.json con el display.

Wokwi web

Save a copy. En diagram.json: borrar todo, pegar diagram_7seg.json y **Save**.

Placa real

No hay display: el patrón se ve en los **8 LEDs**. Con práctica, se aprende a "leer" los dígitos.

En la teoría, el **7447/7448** se diseña con 7 mapas de Karnaugh. Acá la tabla de verdad **es** el programa: la simplificación ahorra compuertas; en una memoria, todas las tablas ocupan lo mismo.

El selector es el índice

S1 (B)	S0 (A)	Y
0	0	D0
0	1	D1
1	0	D2
1	1	D3

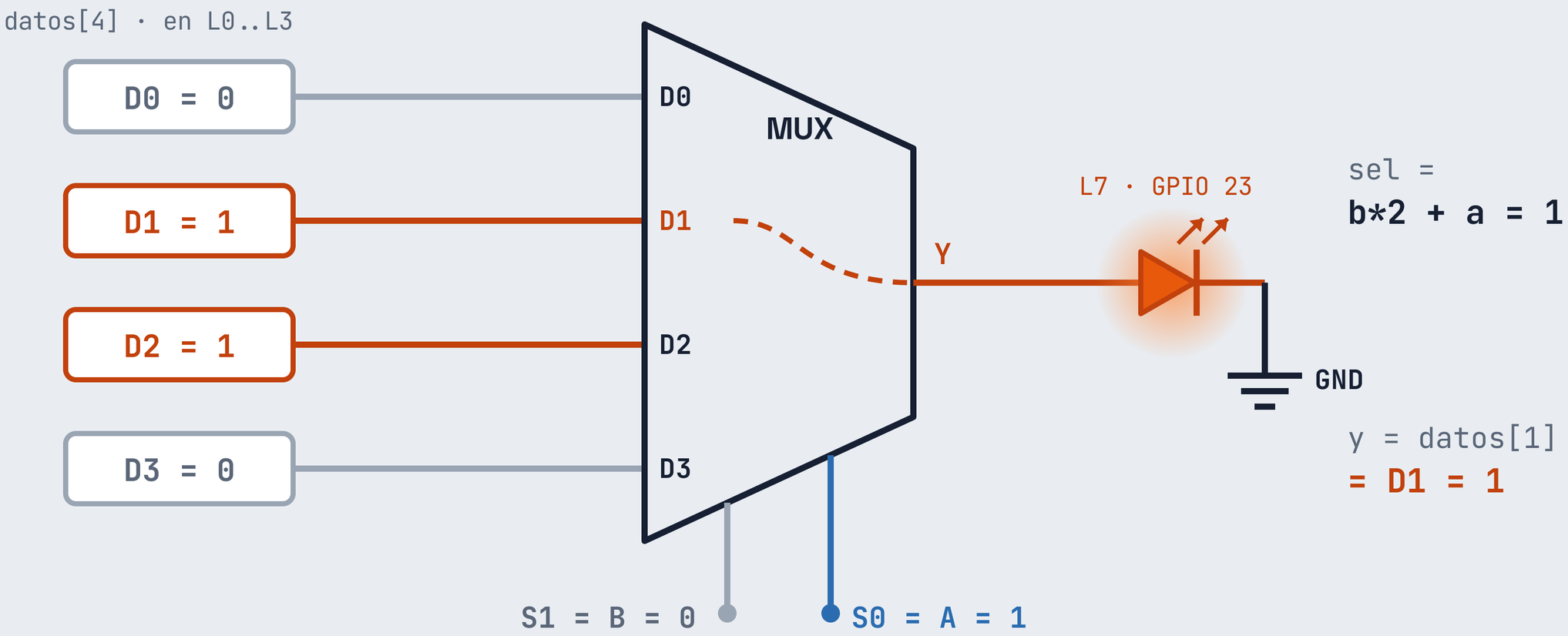
```
bool datos[4] = {0, 1, 1, 0};   // D0..D3

int sel = b * 2 + a;
bool y = datos[sel];       // ¡el MUX entero!
digitalWrite(LEDS[7], y);
```

Datos en **L0..L3**, selector con **A y B**, salida en **L7**.

En el decodificador la tabla es **fija** y el índice elige la salida. En el MUX los **datos** cambian y el índice elige **cuál pasa**. Indexar un arreglo es multiplexar.

Selector en 1: pasa D1 a L7



Cualquier función, un arreglo de 8

```
//                  fila: 0  1  2  3  4  5  6  7
const bool F[8] =       {0, 1, 1, 0, 1, 0, 0, 1};

bool a = (digitalRead(BOTON_A) == LOW);
bool b = (digitalRead(BOTON_B) == LOW);
bool c = (digitalRead(BOTON_C) == LOW);
int fila = c * 4 + b * 2 + a;
digitalWrite(LEDS[0], F[fila]);
```

Esta es **A ^ B ^ C**, la paridad impar. El programa no lo sabe: solo copia la columna F.

`{0, 0, 0, 1, 0, 1, 1, 1}`

Mayoría: sin tocar el código, solo la tabla.

$2^8 = 256$

funciones de 3 variables en una ROM de 8 × 1.

8 · EJERCICIOS · 40 MIN

Ahora ustedes

1 · BCD con error

Tabla de 16: las entradas 10..15 muestran una **E**. Recorrer las 16 cada 800 ms. Sin if: la decisión está en la tabla.

2 · Codificador de prioridad

Posición del 1 de más peso (0..7) o -1. Buscar con for desde el bit 7. **0b00101000** da 5.

3 · Código Gray

Tabla **GRAY[16]** contra **n ^ (n >> 1)**: contar diferencias (tienen que ser 0). Binario en L4..L7, Gray en L0..L3.

4 · Comparador de 2 bits

X con A, Y con B. L5 si X < Y, L6 si X == Y, L7 si X > Y. Tabla de 16 indexada con **y * 4 + x**.

TP CLASE 6

El entrenador decodifica

1. Comentario de encabezado: nombre, clase y qué hace.
2. Una variable valor que va de 0 a 15.
3. A suma 1 y B resta 1, con vuelta: después de 15 viene 0 y antes de 0 viene 15. Con delay(250).
4. Mostrar valor en el display con SEGMENTOS[16] y mostrarByte.
5. Cada vez que cambia, imprimir exactamente así (con imprimirBinario).
6. Al arrancar, un cartel y el valor inicial (0).

Entrega: carpeta **SL2026-Apellido-Clase6** en ZIP, sin la carpeta .pio.

Partir del TP de la Clase 5 y usar la placa con display: **placa/diagram_7seg.json**.

```
Valor: 11 (0xB) → segmentos 0b01111100
```

CHECKLIST DE SALIDA

Al terminar la clase, sé…

- declarar e inicializar un arreglo, y decir cuáles son sus índices válidos
- recorrer un arreglo con for para sumar, promediar y buscar el máximo
- explicar qué pasa en C si se usa un índice fuera del arreglo
- explicar por qué un combinacional es una tabla (entrada = índice, salida = contenido)
- armar el índice a partir de varias entradas: c * 4 + b * 2 + a
- hacer un decodificador, un multiplexor y una función de 3 variables con arreglos
- armar el patrón de 7 segmentos de un dígito y usar el display en Wokwi

La semana que viene: hoy la salida dependía solo de lo que apretaban ahora. En la Clase 7 el micro va a **recordar lo que pasó antes**: flip-flops y contadores.